DATA COUCH

Real-Time Streaming & Lakehouse Pipelines

Redpanda Platform & Operations (Fundamentals)

AGENDA

- Redpanda Overview
- Core Kafka Concepts
- Redpanda Architecture
- Replication
- Performance & Efficiency
- Compatibility & Operational Concepts

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

What is Redpanda?

Redpanda is a high-performance streaming data platform, fully compatible with **Kafka APIs** but simpler to run, faster, and does not require ZooKeeper.

- Low latency and high throughput.
- Kafka API compatible (no code changes needed).
- Simplified architecture (no ZooKeeper).
- uns anywhere – on-premises, cloud, or containers.

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

The Next Generation of Streaming

Introducing Redpanda

The Goal

Build a true Kafka replacement that addresses the operational and performance drawbacks of the legacy architecture

Engineered from Scratch

Not an evolution of Kafka, but a completely new system

Core Focus Areas:

1. *Ultra-Low, predictable latency*
2. *Operational simplicity (Single Binary)*
3. *Lower total cost of ownership (TCO)*

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Recap Core Concept:

History of Streaming

The History of Streaming

Before Apache Kafka

- Traditional Message Queues (e.g., RabbitMQ, ActiveMQ)
- Focus on message delivery guarantee and consumption tracking
- Messages often deleted upon consumption
- Not designed for stream replay or retention at scale

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

The Fundamental Shift

Batch Processing

- **Definition:** Operates on large, *static* datasets (data at rest)
- **Tools:** Hadoop, Spark Batch
- **Latency:** High (minutes to hours)
- **Use Case:** End-of-day reports, historical analysis

Streaming Processing

- **Definition:** Handles *unbounded*, continuous data flows (data in motion)
- **Tools:** Kafka, Redpanda, Flink
- **Latency:** Low (ms to seconds)
- **Use Case:** Real-time dashboards, fraud detection, IoT telemetry

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

The Evolution: Kafka's Genesis

2008

The LinkedIn Problem

LinkedIn's systems struggled to track user activity at scale as millions of profile views, clicks, and messages were generated every second—far beyond what traditional pipelines could handle. Legacy queues like ActiveMQ and RabbitMQ couldn't meet the required throughput, durability, or fault tolerance.

2010

The Birth of Kafka

LinkedIn engineers Jay Kreps, Neha Narkhede, and Jun Rao began developing Kafka—a distributed system designed to process massive event streams with high throughput and low latency. Inspired by database log abstractions and scalable publish-subscribe models, Kafka aimed to unify online and offline data pipelines into a single, reliable streaming platform.

2011

Kafka is Open-Sourced

LinkedIn open-sourced Apache Kafka, bringing its powerful streaming technology to the world. Kafka introduced a publish-subscribe architecture built for massive scale, durable log-based storage for replay and recovery, and horizontal scalability across distributed clusters—setting a new standard for real-time data streaming.

2020

Streaming Becomes the Standard

Real-Time data streaming had become a foundation of modern infrastructure, powering fintech, e-commerce, IoT, and AI systems. Kafka's open architecture inspired next-generation platforms like Redpanda, which reimagines Kafka for the cloud era — offering the same API compatibility, but with a faster, simpler, and cloud-native design, free from JVM, ZooKeeper, and heavy operational overhead.

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

The Operational Challenge of Traditional Kafka

The Legacy Stack

JVM Heritage

- Kafka runs on the Java Virtual Machine (JVM)
- Leads to unpredictable performance due to **Garbage Collection (GC) pauses**
- Requires significant memory overhead and complex tuning

ZooKeeper Dependency

- Kafka required an external, separate cluster (ZooKeeper) for metadata, cluster coordination, and controller election
- This is known as the dual-system problem, adding significant operational load

Scaling Limitations

- Complexity in scaling the separate ZooKeeper and Kafka clusters

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

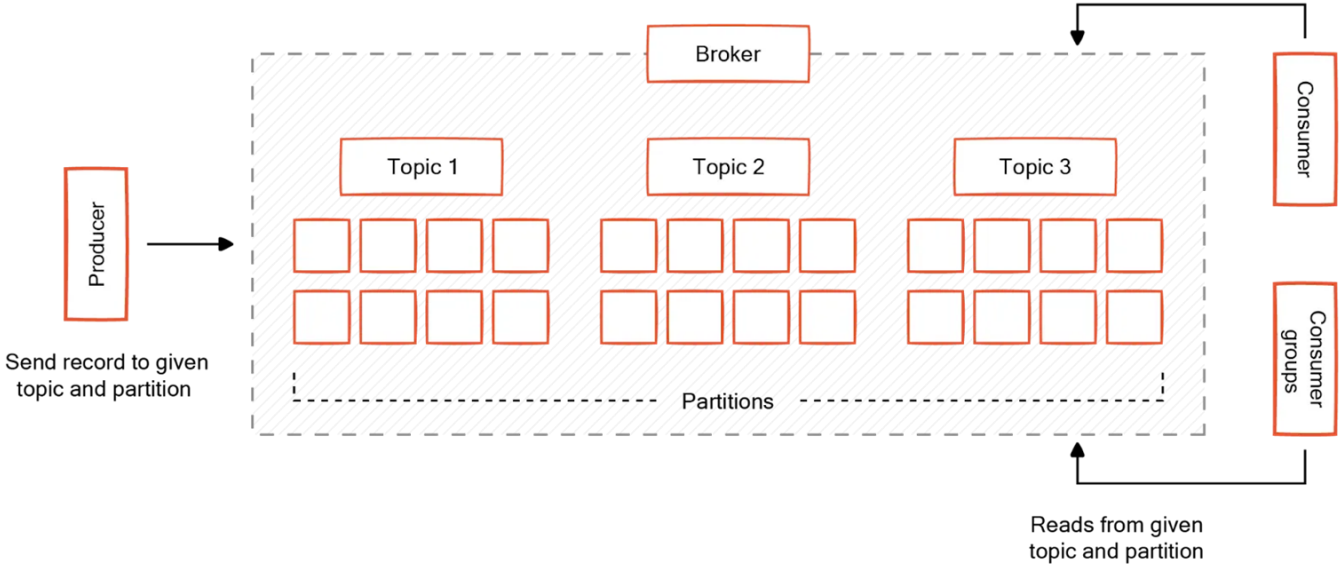
Kafka basics

Topics	*Logical channels where messages are published*
Partitions	*Topics are split into partitions for scalability and parallelism*
Producer	*Applications that publish (write) data to topics*
Consumer	*Applications that subscribe (read) data from topics*
Offset	*Unique sequence numbers within partitions that track message position*

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Kafka Topics

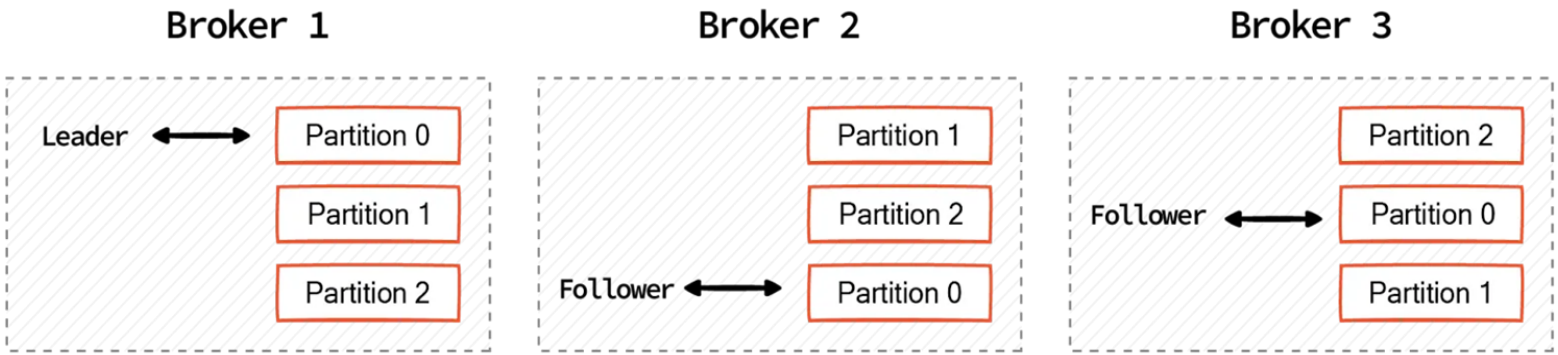
A Kafka topic is a logical channel or stream where messages (events) are published by producers and consumed by consumers.



Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Kafka Partitions

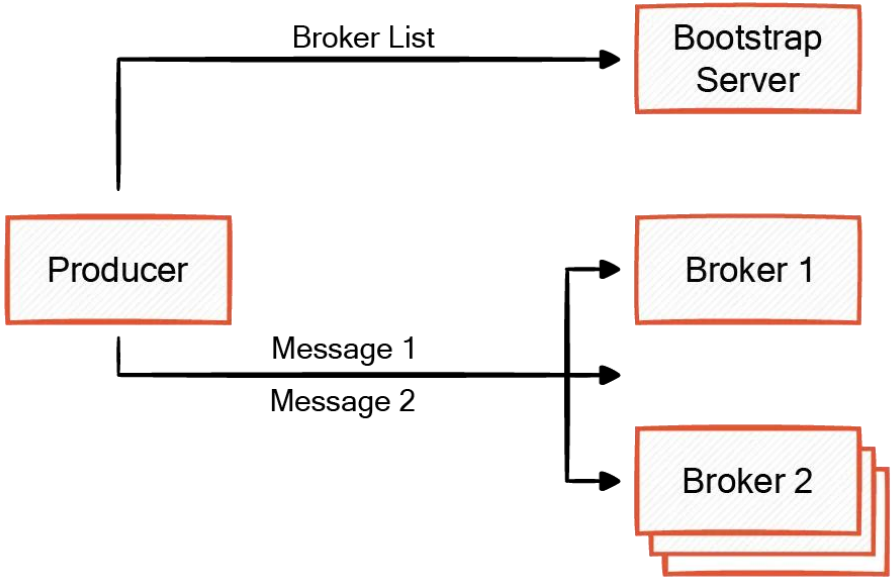
Partitions are ordered, append-only logs that store subsets of a topic's data. Multiple partitions allow Kafka to process messages in parallel



Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Kafka producers/consumers

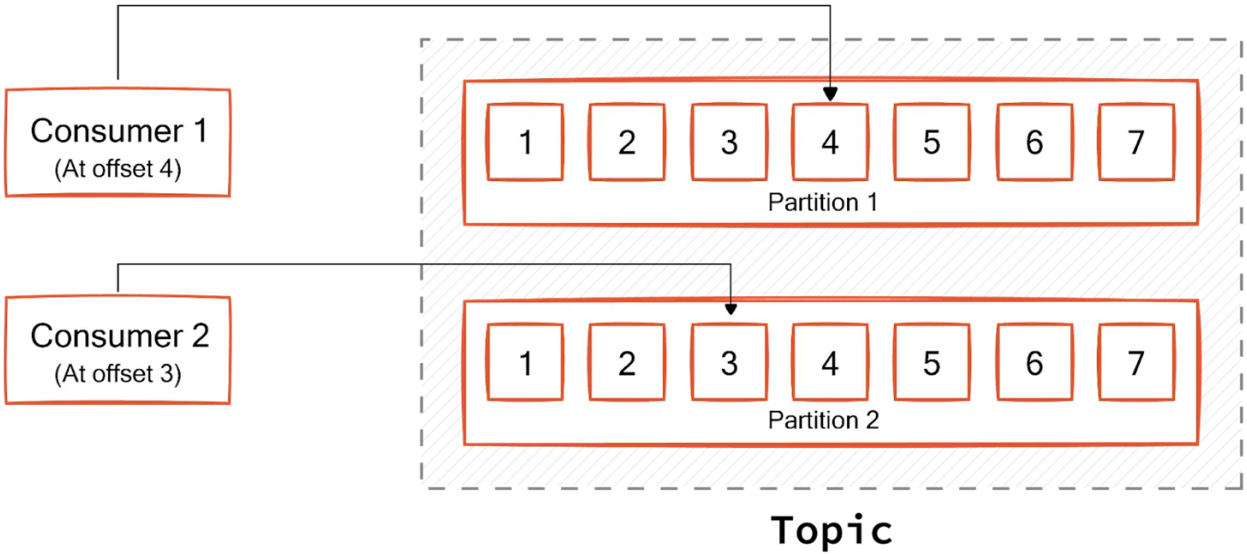
When processing data using Apache Kafka's architecture, components connected to Kafka are producers or consumers. Producers store messages (or events) in Kafka, and consumers read these messages.



Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Kafka Offset

Offset is a continuously increasing identifier that represents the order of a message from the beginning of the partition.



Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Redpanda Architecture

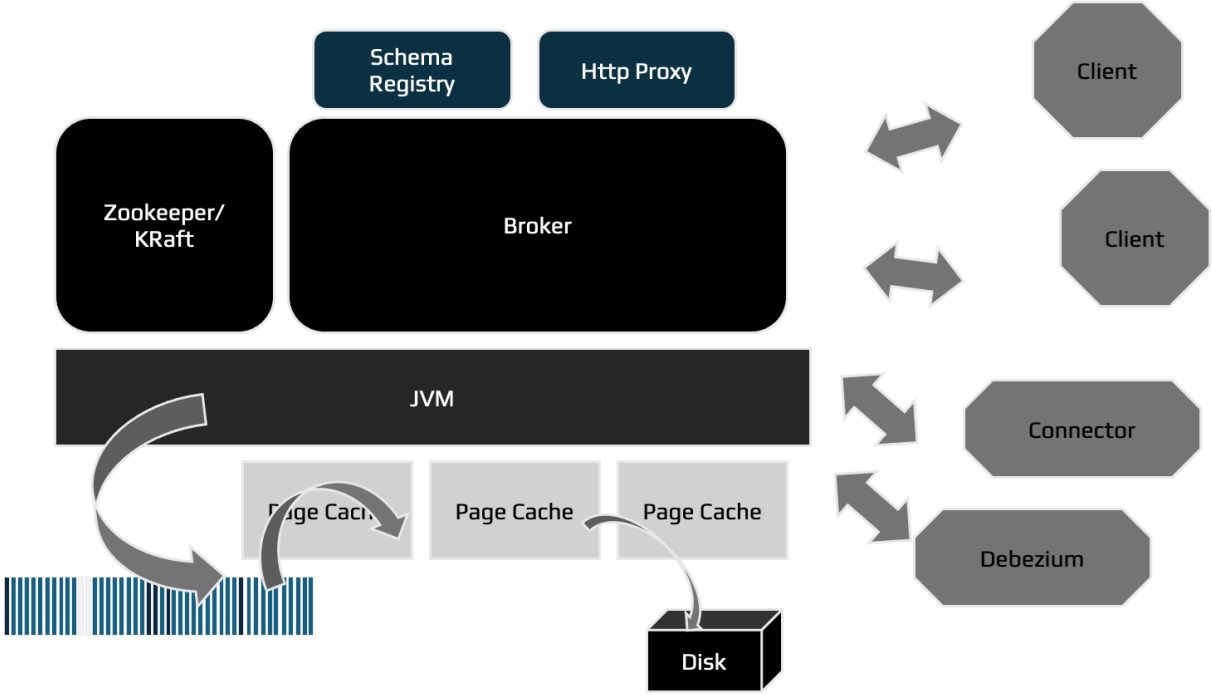
Core Architectural Differentiators

Redpanda was re-written from the ground up to eliminate the performance bottlenecks and operational overhead of Kafka's traditional architecture.

Feature	Apache Kafka (Traditional)	Redpanda (Modern)
Language	Java (JVM)	C++
Framework	Blocking I/O, OS Scheduler	Seastar (Non-blocking, Event-driven)
Consensus	ZooKeeper (Separate)	Raft (Built-in)
Concurrency	Multi-threaded	Thread-per-Core

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

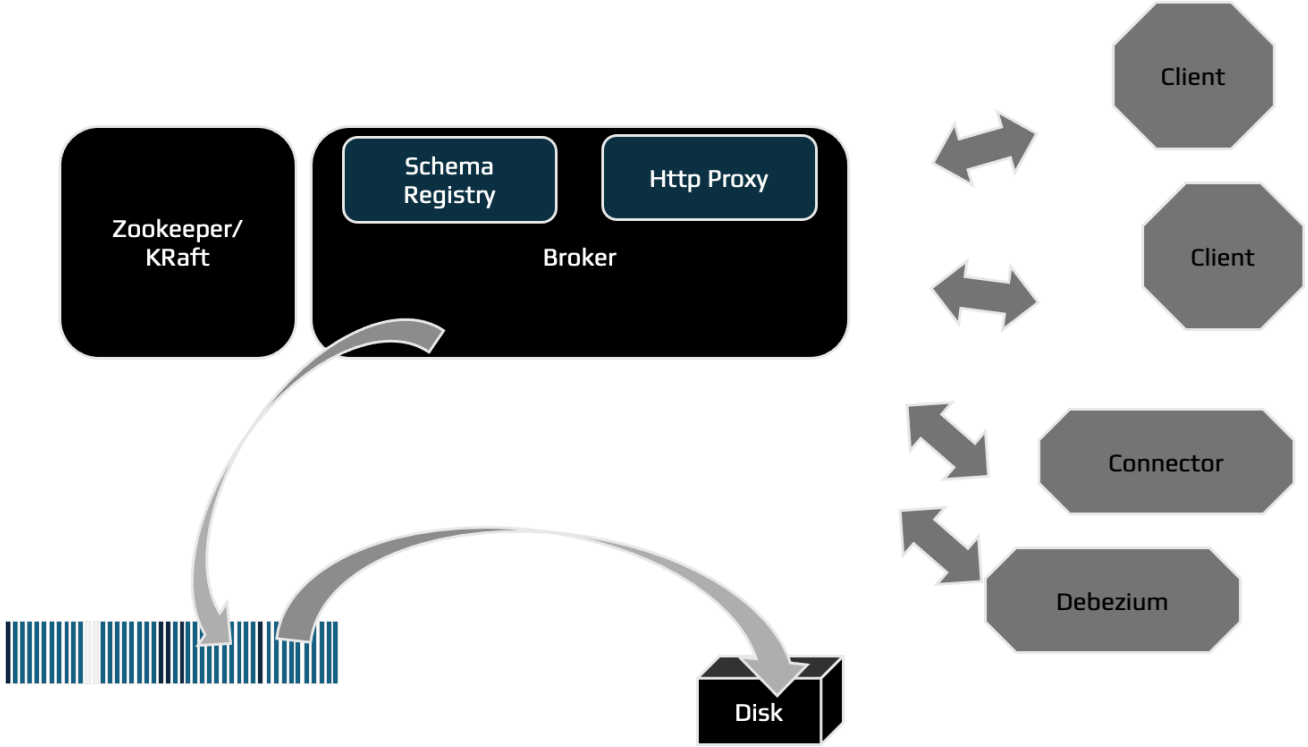
Apache Kafka Architecture



Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Redpanda Broker architecture

Simple, powerful, cost-efficient, reliable



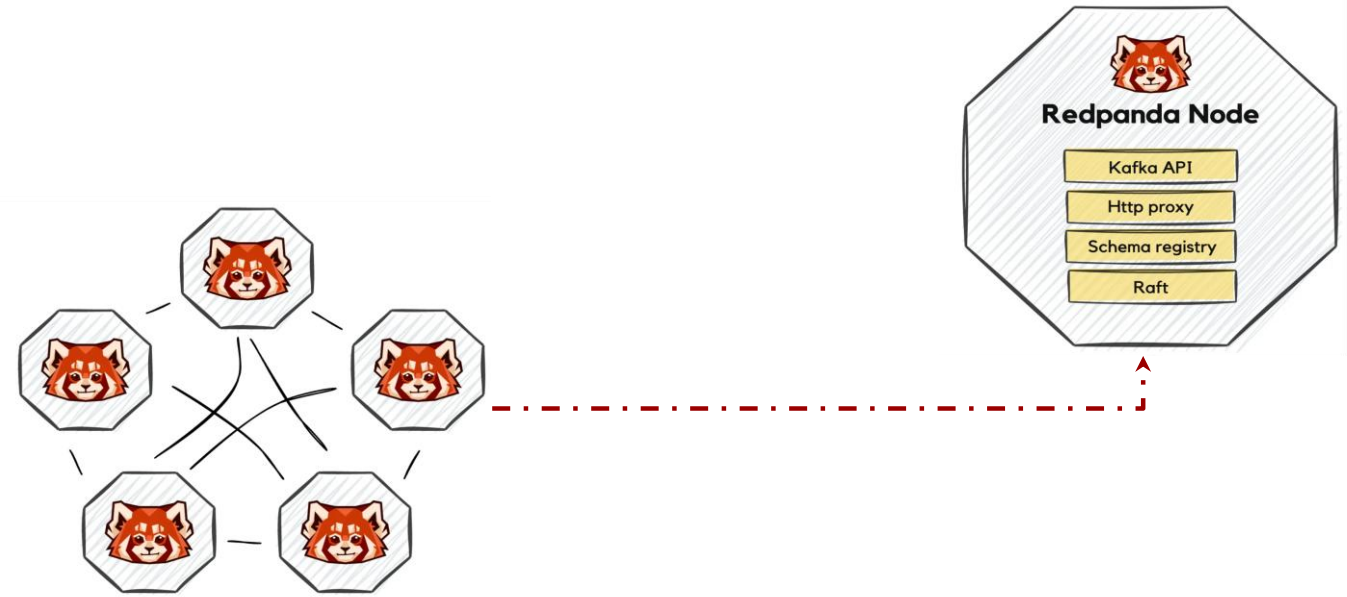
Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Single binary design

Redpanda is distributed as a single executable binary.

Unlike Kafka:

which needs JVM, ZooKeeper, and multiple configs, Redpanda packages everything in one binary.



Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Thread-per-core model

Redpanda uses a thread-per-core architecture, Instead of many threads competing across CPUs , each CPU core runs its dedicated thread, These threads communicate via message passing instead of locks.

Benefits:

- **Ultra-low latency** → *avoids lock contention and CPU bottlenecks*
- **Predictable performance** → *each core works independently*
- **High throughput** → *can process millions of messages/sec efficiently*

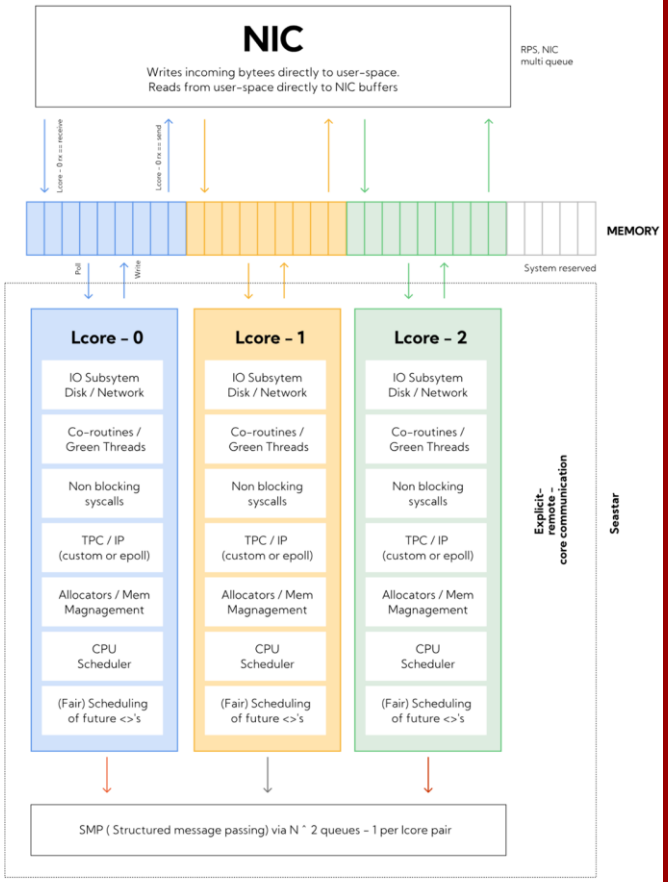
Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Dedicated Resources, Predictable Performance

- Redpanda dedicates specific CPU cores to specific I/O and networking tasks.
- Each core manages its own memory and processes its own share of partitions/requests.

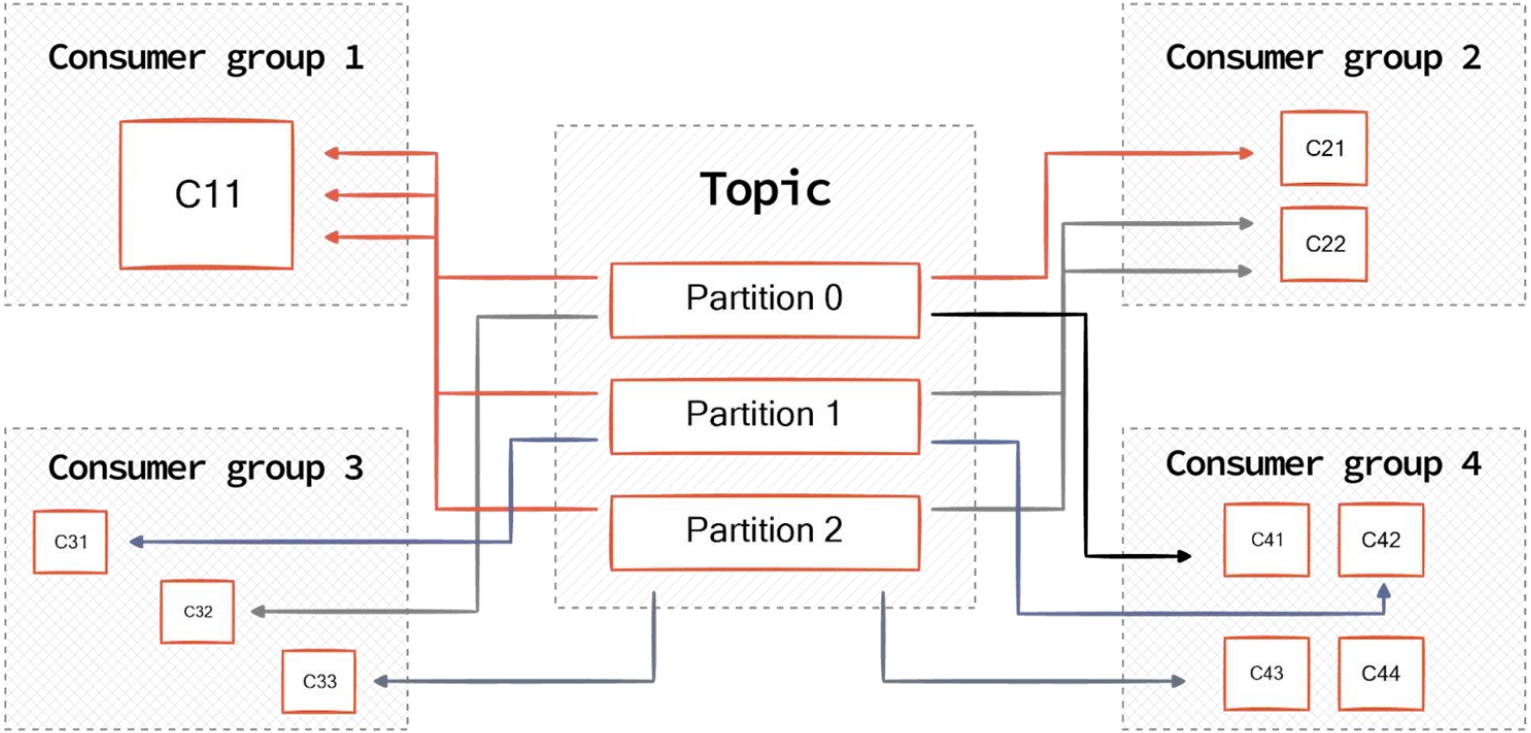
Key advantages:

1. ***Cache Locality:*** *Data required for processing stays within the core's cache.*
2. ***No Context Switching:*** *Eliminates expensive overhead of the OS scheduler moving threads.*
3. ***Vertical Scaling:*** *Performance scales near-linearly as you add more cores.*



Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Partition Assignment



Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

The Problem with ZooKeeper

The Dual-System Complexity

- **External Dependency:** ZooKeeper is a completely separate technology stack (Java) that must be managed alongside Kafka.
- **Operational Burden:** Requires separate maintenance, monitoring, backups, and scaling procedures.
- **Split Brain Risk:** A failure in ZK can destabilize the Kafka cluster controller, potentially leading to incorrect metadata states.
- **Slow Recovery:** Metadata changes (like leader elections) must propagate through ZooKeeper, adding latency to failover events.

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

ZooKeeper-less Design & Raft Consensus

Simplified Cluster State Management

- **Redpanda Solution:** Integrated metadata management.
- **How:** Redpanda replaced the external ZooKeeper dependency with a **built-in Raft Consensus Engine**.
- **Result:** The entire system—data and metadata—is managed within the Redpanda brokers themselves.
- **Benefit:** Operators only have one single set of processes to deploy, monitor, and scale.

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Core Building Blocks

Replication	*Data copied across brokers for durability*
Partitions	*Enable parallelism + fault isolation*
Log Compaction	*Keeps latest state while saving storage*
High Availability	*Cluster-level fault tolerance*

Mini quiz:
"If RF=2 and 1 broker fails, do we lose data?"

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Replication & HA Overview

Kafka ISR vs. Redpanda Raft

While both systems ensure data is replicated, the underlying mechanisms differ significantly.

Concept	Apache Kafka	Redpanda (Raft)
ISR Tracking	Controller-driven. ISR is a *subset* of followers caught up to the leader's High Water Mark.	Raft-driven. Replication quorum is strictly managed by Raft consensus.
Leader Election	Handled by ZooKeeper.	Handled by the internal Raft engine.
Consistency	Achieved by ISR consensus.	Achieved by Raft log commitment.

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

In-Sync Replica Management

1. In Redpanda, the set of replicas in the Raft group is always tightly managed.
2. A write is only committed after it has been acknowledged by a majority (quorum) of the replicas.
3. This inherent quorum-based system naturally prevents split-brain scenarios and ensures strong consistency.

Benefit: *Faster, more deterministic leader failover times compared to ZooKeeper-based sys*

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

The Single Source of Truth

1

- As in Kafka, every topic partition in Redpanda has one designated Leader.

2

- Producers must route their requests to the Leader for a given partition.

3

- Followers continuously pull data from the Leader to ensure their logs are synchronized.

Goal: Ensure a strict global ordering for messages within that partition.

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

HA features in Redpanda

Replica Synchronization

Ensures data consistency using Raft quorum-based replication.

Rack Awareness

Distributes replicas across racks or zones to avoid single points of failure.

Partition Leadership

Raft heartbeat and leader elections maintain availability during failures.

Producer Acknowledgment

acks define the durability-latency trade-off in message delivery.

Partition Rebalancing

Automatically redistributes partitions when brokers are added, removed, or fail.

Tiered Storage & Disaster Recovery

Provides long-term retention and fast recovery of data after failures.

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

performance insights

Redpanda vs Apache Kafka performance insights

Redpanda

Ultra-low, stable latency (up to 70× faster), breaks 1 GB/s on just 3 nodes, efficient even on small ARM instances.

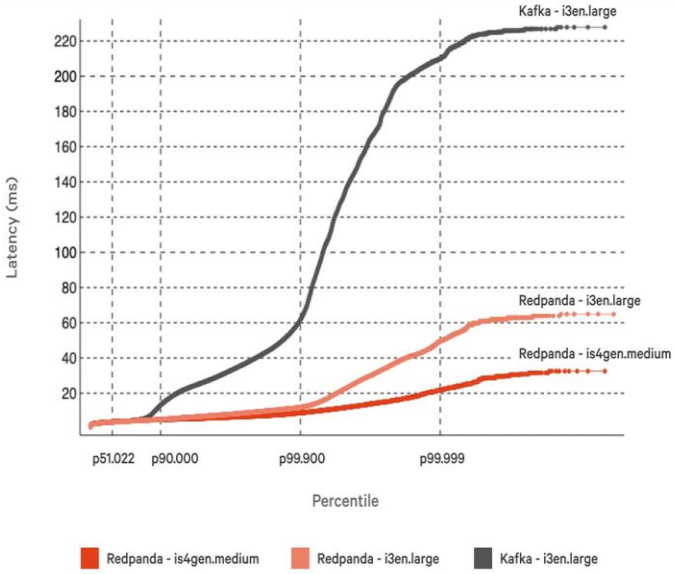
Apache Kafka

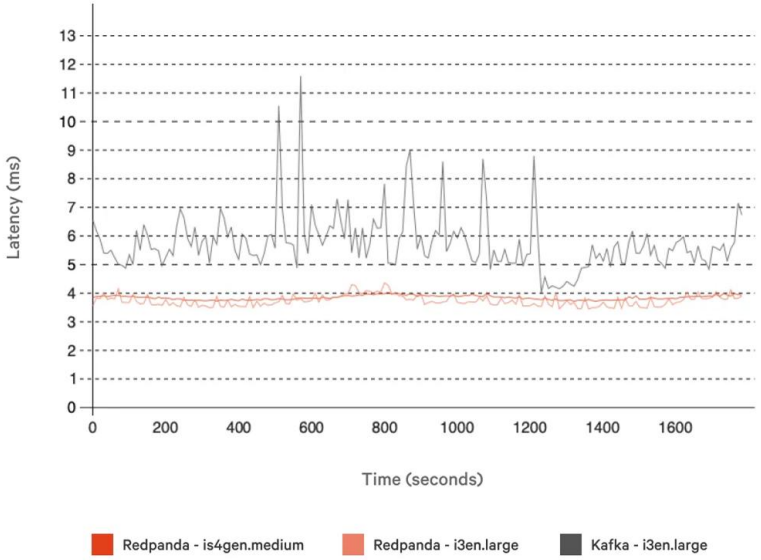
Higher latency, needs 2–3× more nodes and hardware to reach similar throughput, struggles on ARM.

Workload size/Throughput	Kafka P99.99	Redpanda P99.99	Redpanda faster by
Small - 50MB/s	*164.57ms (3 nodes)*	*13.91ms (3 nodes)*	*12x*
Medium - 500MB/s	*388.66ms (4 nodes)*	*16.658ms (3 nodes)*	*23x*
Large - 1GB/s	*5509.73ms (6 nodes)*	*79.574ms (3 nodes)*	*70x*

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Comparing performance at 50MB/sec





50MB/sec: *End-to-End Latency*

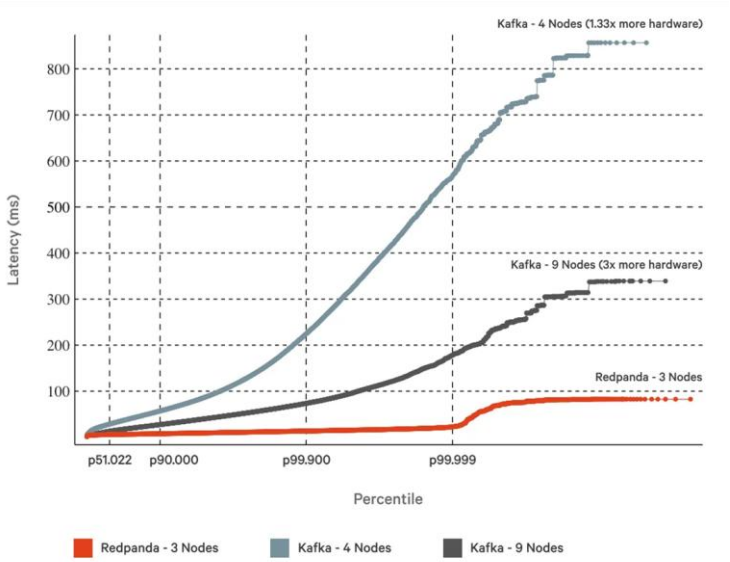
Percentiles: *lower is better*

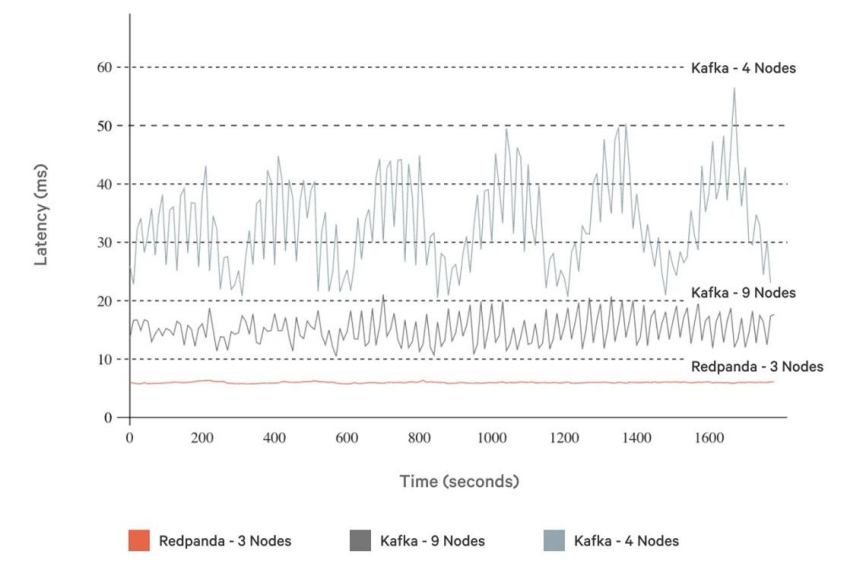
50MB/sec: *End-to-End Latency*

Average: *lower is better*

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Comparing performance at 500MB/sec





500MB/sec: *End-to-End Latency*

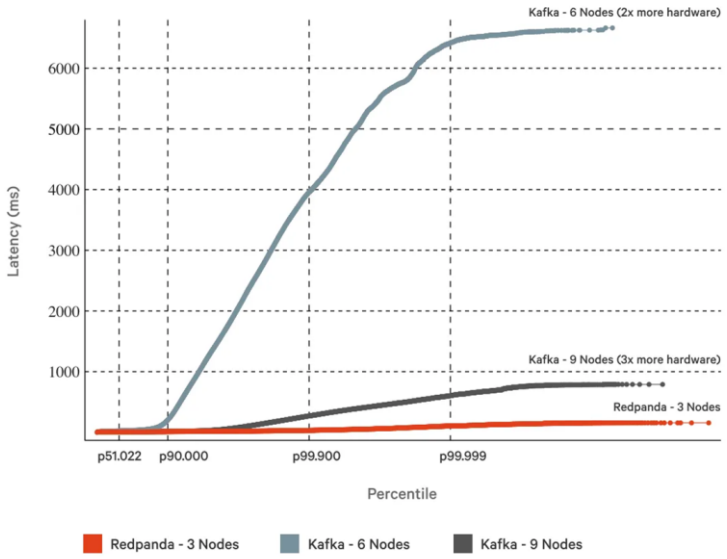
Percentiles: *lower is better*

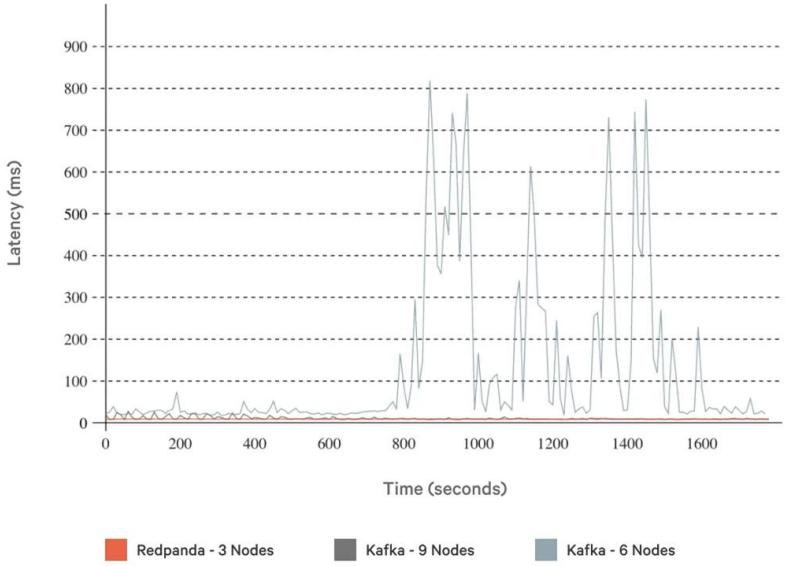
500MB/sec: *End-to-End Latency*

Average: *lower is better*

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Comparing performance at 1GB/sec





1GB/sec: *End-to-End Latency*

Percentiles: *lower is better*

1GB/sec: *End-to-End Latency*

Average: *lower is better*

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Compatibility & Operational Concepts

Kafka Protocol Compatibility

The single most critical feature of Redpanda is its wire-level compatibility with the Apache Kafka protocol

- *This is not just "similar," but a **true implementation of the Kafka API** specification.*
- *Redpanda accepts, processes, and responds using the exact same request/response binary format as a Kafka broker.*

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Connecting Your Existing Application

Changing your application from Kafka to Redpanda is usually a configuration swap:

Property	Kafka Value	Redpanda Value
bootstrap.servers	kafka-broker-1:9092	redpanda-broker-1:9092

***That's it**. The application code, including serialization/deserialization logic, remains untouched.*

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

The Role of Keyed Messages

Identical to Kafka

The mechanism and purpose of using message keys remain unchanged when switching to Redpanda.

Messages consist of:

1. **Topic:** The log to which the message is written.
2. **Key:** (Optional) A byte array used for partitioning.
3. **Value:** The message payload.
4. **Headers:** (Optional) Metadata.

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Guaranteeing Message Order

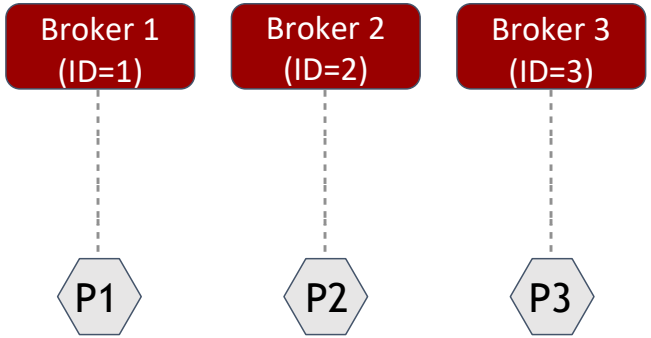
- The message key determines which partition the message is written to.
- Default Behavior (Kafka & Redpanda): Messages with the same key are always routed to the same partition.
- Ordering is guaranteed only within a single partition.

Example: All messages about User-123 will land on Partition 4, and they will be processed sequentially

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Redpanda Broker IDs

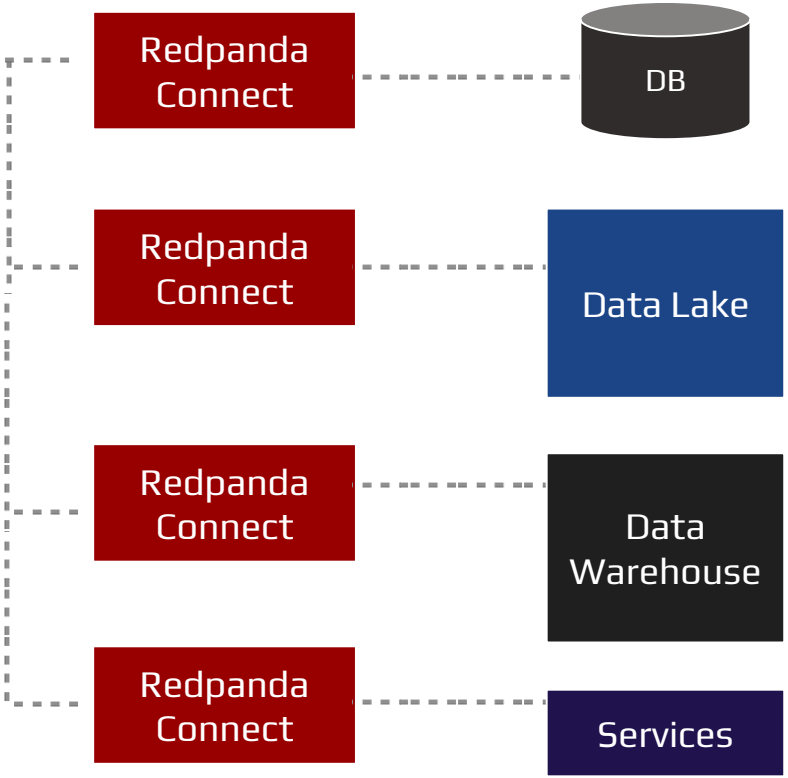
- A **Broker ID** is a **unique identifier** assigned to each broker (server/node) in the Redpanda cluster.
- It helps the cluster **distinguish one broker from another**.
- Even if a broker restarts, the same ID ensures the cluster recognizes it as the same node.



Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

Redpanda Connect

Redpanda Connect is a framework for moving data in and out of Redpanda without writing custom code.



Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

ANY QUESTIONS?

Copyright © 2026 by DataCouch Pvt. Ltd.. All rights reserved.

THANK YOU